

Nicolas DARNIS

Forme normale de Chomsky et algorithme CYK

Rapport de projet

Année 1998-99

Table des matières

Introduction	1
1 Les grammaires algébriques et leurs standardisations	3
1.1 Distribution	3
1.2 Remplacement de $=$ par $\supseteq$	3
1.3 Elimination de l'union	4
1.4 Transformation du membre droit de chaque règle en une lettre ou en une concaténation de variables	4
1.5 Elimination des membres droits trop longs	5
1.6 Elimination d'autant de mots vides que possible	5
1.7 Elimination des règles unitaires	8
2 L'algorithme CYK	11
3 Réalisation	13
3.1 Structures de données	13
3.2 Saisie de la grammaire	14
3.3 Standardisation et mise en forme normale de Chomsky de la grammaire G saisie . .	15
3.3.1 Elimination de l'union	16
3.3.2 Transformation du membre droit de chaque règle en une lettre ou en une concaténation de variables	16
3.3.3 Elimination des membres droits trop longs	17

3.3.4	Elimination d'autant de mots vides que possible	17
3.3.5	Elimination des règles unitaires	19
3.4	Algorithme CYK	20
3.5	Utilisation	20
Conclusion		23
A	Algorithme déterminant si $\forall V, \varepsilon \in L(V)$	25
B	Algorithme déterminant si une variable A conduit à une variable B	27
C	Quelques exemples	29
C.1	Exemple a: le langage ETF	29
C.1.1	Fichier décrivant la grammaire	29
C.1.2	Forme normale de Chomsky	29
C.2	Exemple b: le langage des mots de parenthèses sur $\{a, b\}$	30
C.2.1	Fichier décrivant la grammaire	30
C.2.2	Forme normale de Chomsky	30
C.3	Exemple c: une variante du langage des mots de parenthèses sur $\{a, b\}$	31
C.3.1	Fichier décrivant la grammaire	31
C.3.2	Forme normale de Chomsky	31
C.4	Exemple d: le langage des palindromes sur $\{a, b\}$	32
C.4.1	Fichier décrivant la grammaire	32
C.4.2	Forme normale de Chomsky	32
Bibliographie		33

Introduction

Le but de ce projet est de réaliser un algorithme de mise en forme normale de Chomsky d'une grammaire algébrique G (voir [AA91]), ainsi qu'un algorithme de reconnaissance d'un mot appartenant à $L(G)$, le langage engendré par G .

Nous allons tout d'abord préciser les notations utilisées dans la suite de ce rapport. Une grammaire algébrique G est composée de symboles terminaux appelés *lettres* dont l'ensemble sera noté Σ ainsi que de symboles non terminaux, appelés *variables*, dont l'ensemble sera noté V dans toute la suite. Par convention nous noterons les lettres en minuscules, et les variables en majuscules. Enfin, le mot vide sera noté ε .

Mise en forme normale de Chomsky

L'algorithme de mise en forme normale de Chomsky d'une grammaire algébrique que nous avons réalisé, repose sur la méthode décrite dans [eRB94]. Nous détaillons cette méthode au Chapitre 1.

Reconnaissance d'un mot

L'algorithme de reconnaissance d'un mot appartenant au langage algébrique engendré par une grammaire algébrique en forme normale de Chomsky que nous avons réalisé, repose sur la méthode de Cocke, Younger et Kasami décrite dans [eRB94]. Cet algorithme est détaillé au Chapitre 2.

Chapitre 1

Les grammaires algébriques et leurs standardisations

Nous allons présenter ici une méthode pour mettre un système d'équations algébriques sous une forme standardisée que l'on appellera une grammaire algébrique. Nous standardiserons ensuite encore les grammaires algébriques pour les mettre sous une forme normale due à Chomsky, et qui a été utilisée dans les premiers algorithmes efficaces d'analyse syntaxique. Notre objectif sera de mettre tout système d'équations sous une forme particulièrement simple.

1.1 Distribution

Comme la concaténation se distribue sur l'union, le membre droit d'une équation algébrique peut être écrit comme une union de concaténations. Considérons par exemple l'équation algébrique

$$L = \varepsilon \cup a(\varepsilon \cup La) \cup b(\varepsilon \cup Lb) ,$$

qui définit le langage des palindromes sur $\Sigma = \{a, b\}$. Par distributivité, cette équation peut se réécrire sous la forme

$$L = \varepsilon \cup a \cup aLa \cup b \cup bLb .$$

1.2 Remplacement de $=$ par $\supseteq$

Théorème 1 (*Théorème de Tarski-Knaster*). *Si f est une fonction monotone de $\mathcal{D}$ dans $\mathcal{D}$, alors l'inéquation $f(X) \prec X$ a une plus petite solution P . De plus, P est aussi l'unique solution minimale de l'équation $f(X) = X$. Autrement dit, P est l'unique plus petit point fixe de f .*

Le théorème de Tarski-Knaster montre que les solutions minimales des systèmes d'équations sont les plus petits ensembles qui se contractent sous l'action du système: cela justifie le fait de pouvoir remplacer l'égalité $=$ par une inégalité $\supseteq$ dans un système d'équations algébriques. L'exemple précédent devient donc

$$L \supseteq \varepsilon \cup a \cup aLa \cup b \cup bLb .$$

1.3 Elimination de l'union

Pour éliminer l'union, il suffit de remplacer une inéquation du type $V \supseteq f \cup g$ par deux inégalités $V \supseteq f$ et $V \supseteq g$ et d'itérer ce processus si besoin est. Cette transformation donne donc sur l'exemple précédent:

$$\begin{aligned} L &\supseteq \varepsilon \\ L &\supseteq a \quad L \supseteq aLa \\ L &\supseteq b \quad L \supseteq bLb . \end{aligned}$$

Un système d'équations algébriques qui se trouve sous une telle forme est appelé une *grammaire algébrique* pour L . Nous nous intéresserons typiquement à résoudre un tel système d'inégalités pour seulement l'une des variables, disons S , qui est appelée la variable *initiale* (ou *axiome*) de la grammaire. Si G est une grammaire algébrique dont l'axiome est S , nous désignerons par $L(G)$ au lieu de $L(S)$ le langage *engendré* par G . On dit parfois que les mots de G sont des phrases. Un langage engendré par une grammaire algébrique est appelé un *langage algébrique*.¹ La classe des langages algébriques est fermée sous les opérations rationnelles. Quand on décrit les grammaires algébriques, on remplace en général les inclusions par des flèches allant de la gauche vers la droite qui sont appelées des règles (ou des productions car on peut en effet penser que la variable de gauche produit le résultat de droite).

Exemple: La grammaire suivante est une grammaire algébrique pour le langage des palindromes sur $\Sigma = \{a, b\}$:

$$\begin{aligned} L &\rightarrow \varepsilon \\ L &\rightarrow a \quad L \rightarrow aLa \\ L &\rightarrow b \quad L \rightarrow bLb \end{aligned}$$

On parle de la variable (resp. de la suite de variables et de lettres) qui apparaît à gauche (resp. à droite) d'une flèche comme du membre gauche (resp. droit) de la règle. Les règles de la forme $X \rightarrow Y$ où Y est une variable sont appelées des *règles unitaires* et celles de la forme $X \rightarrow \varepsilon$ des *règles vides*.

Le reste de ce paragraphe est consacré à la présentation de la mise sous forme normale de Chomsky d'une grammaire algébrique. Cette standardisation introduit de nouvelles variables mais ne modifie pas les solutions associées aux anciennes.

1.4 Transformation du membre droit de chaque règle en une lettre ou en une concaténation de variables

Pour chaque lettre c , introduisons une nouvelle variable V_c et la règle $V_c \rightarrow c$. Dans chacune des autres règles, remplaçons c par V_c . Ce système (sauf pour les nouvelles variables) a bien les mêmes

¹On utilise aussi parfois la terminologie "indépendant du contexte", traduction littérale du terme anglais "context-free". Le terme français d'"algébrique" met l'accent sur le fait que les langages algébriques sont exactement les solutions des analogues non commutatifs des systèmes d'équations algébriques commutatives usuelles.

solutions que l'ancien. L'exemple que nous avons utilisé jusqu'à présent se transforme en

$$\begin{aligned}
 L &\rightarrow \varepsilon \\
 L &\rightarrow V_a \\
 L &\rightarrow V_a L V_a \\
 L &\rightarrow V_b \\
 L &\rightarrow V_b L V_b \\
 V_a &\rightarrow a \\
 V_b &\rightarrow b .
 \end{aligned}$$

1.5 Elimination des membres droits trop longs

Remplaçons chaque règle de la forme $X \rightarrow Y_1 Y_2 \cdots Y_k$, avec $k \geq 3$ par les règles suivantes:

$$\begin{aligned}
 X &\rightarrow Y_1 N_1 \\
 N_1 &\rightarrow Y_2 N_2 \\
 &\vdots \\
 N_{k-3} &\rightarrow Y_{k-2} N_{k-2} \\
 N_{k-2} &\rightarrow Y_{k-1} Y_k ,
 \end{aligned}$$

où $N_1, \dots, N_{k-2}$ sont de nouvelles variables. Le nouveau système a exactement les mêmes solutions (sauf pour les nouvelles variables) que l'ancien. Chaque règle est maintenant de la forme $X \rightarrow \varepsilon$, $X \rightarrow c$, $X \rightarrow Y$ ou $X \rightarrow YZ$. Reprenons l'exemple précédent qui devient donc

$$\begin{aligned}
 L &\rightarrow \varepsilon \\
 L &\rightarrow V_a \\
 L &\rightarrow V_a N_1 \\
 N_1 &\rightarrow L V_a \\
 L &\rightarrow V_b \\
 L &\rightarrow V_b N_2 \\
 N_2 &\rightarrow L V_b \\
 V_a &\rightarrow a \\
 V_b &\rightarrow b .
 \end{aligned}$$

1.6 Elimination d'autant de mots vides que possible

Pour chaque variable X , nous introduisons deux nouvelles variables:

- X_+ qui représentera l'ensemble de tous les mots non vides de X , c'est-à-dire $X - \{\varepsilon\}$.
- X_ε qui représentera l'ensemble des mots vides de X , c'est-à-dire $X \cap \{\varepsilon\}$ (X_ε correspondra donc à $\{\varepsilon\}$ si X contient le mot vide et à $\emptyset$ sinon).

Nous introduisons également les nouvelles règles $X \rightarrow X_+$ et $X \rightarrow X_\varepsilon$. L'exemple précédent devient alors

$$\begin{aligned}
L &\rightarrow \varepsilon \\
L &\rightarrow V_a \\
L &\rightarrow V_a N_1 \\
N_1 &\rightarrow L V_a \\
L &\rightarrow V_b \\
L &\rightarrow V_b N_2 \\
N_2 &\rightarrow L V_b \\
V_a &\rightarrow a \\
V_b &\rightarrow b \\
L &\rightarrow L_+ \\
L &\rightarrow L_\varepsilon \\
V_a &\rightarrow V_{a+} \\
V_a &\rightarrow V_{a\varepsilon} \\
V_b &\rightarrow V_{b+} \\
V_b &\rightarrow V_{b\varepsilon} \\
N_1 &\rightarrow N_{1+} \\
N_1 &\rightarrow N_{1\varepsilon} \\
N_2 &\rightarrow N_{2+} \\
N_2 &\rightarrow N_{2\varepsilon} .
\end{aligned}$$

On remplacera alors chaque règle de la forme $X \rightarrow c$ par $X_+ \rightarrow c$. De même, chaque règle de la forme $X \rightarrow \varepsilon$ sera remplacée par $X_\varepsilon \rightarrow \varepsilon$. On remplacera aussi chaque règle de la forme $X \rightarrow Y$ par $X_+ \rightarrow Y_+$ et $X_\varepsilon \rightarrow Y_\varepsilon$.

Par ailleurs, chaque règle de la forme $X \rightarrow YZ$ sera remplacée par les quatre règles suivantes:

- $X_+ \rightarrow Y_+ Z_+$
- $X_+ \rightarrow Y_+ Z_\varepsilon$
- $X_+ \rightarrow Y_\varepsilon Z_+$
- $X_\varepsilon \rightarrow Y_\varepsilon Z_\varepsilon$.

L'exemple précédent devient alors

$$\begin{aligned}
L_\varepsilon &\rightarrow \varepsilon \\
L_+ &\rightarrow V_{a+} \\
L_\varepsilon &\rightarrow V_{a\varepsilon} \\
L_+ &\rightarrow V_{a+} N_{1+} \\
L_+ &\rightarrow V_{a+} N_{1\varepsilon} \\
L_+ &\rightarrow V_{a\varepsilon} N_{1+} \\
L_\varepsilon &\rightarrow V_{a\varepsilon} N_{1\varepsilon}
\end{aligned}$$

$$\begin{aligned}
N_{1+} &\rightarrow L_+ V_{a+} \\
N_{1+} &\rightarrow L_+ V_{a\varepsilon} \\
N_{1+} &\rightarrow L_\varepsilon V_{a+} \\
N_{1\varepsilon} &\rightarrow L_\varepsilon V_{a\varepsilon} \\
L_+ &\rightarrow V_{b+} \\
L_\varepsilon &\rightarrow V_{b\varepsilon} \\
L_+ &\rightarrow V_{b+} N_{2+} \\
L_+ &\rightarrow V_{b+} N_{2\varepsilon} \\
L_+ &\rightarrow V_{b\varepsilon} N_{2+} \\
L_\varepsilon &\rightarrow V_{b\varepsilon} N_{2\varepsilon} \\
N_{2+} &\rightarrow L_+ V_{b+} \\
N_{2+} &\rightarrow L_+ V_{b\varepsilon} \\
N_{2+} &\rightarrow L_\varepsilon V_{b+} \\
N_{2\varepsilon} &\rightarrow L_\varepsilon V_{b\varepsilon} \\
V_{a+} &\rightarrow a \\
V_{b+} &\rightarrow b \\
L &\rightarrow L_+ \\
L &\rightarrow L_\varepsilon \\
V_a &\rightarrow V_{a+} \\
V_a &\rightarrow V_{a\varepsilon} \\
V_b &\rightarrow V_{b+} \\
V_b &\rightarrow V_{b\varepsilon} \\
N_1 &\rightarrow N_{1+} \\
N_1 &\rightarrow N_{1\varepsilon} \\
N_2 &\rightarrow N_{2+} \\
N_2 &\rightarrow N_{2\varepsilon} .
\end{aligned}$$

Il est facile de vérifier que le système ainsi construit possède exactement (sauf pour les nouvelles variables) les mêmes solutions que l'ancien. Si $L(V)$ désigne la solution minimale associée à la variable V , il n'est pas difficile de voir que l'on aura pour toute variable X

- $L(X) = L(X_+) \cup L(X_\varepsilon)$,
- $L(X_+)$ ne contient pas le mot vide,
- $L(X_\varepsilon)$ ne peut pas contenir de mot non vide.

On aura donc $L(X_\varepsilon) = L(X) \cap \{\varepsilon\}$ et $L(X_+) = L(X) - \{\varepsilon\}$.

On éliminera ensuite toutes les variables originelles X à l'exception de l'axiome S . Les seules règles où S apparaît seront alors $S \rightarrow S_+$ et $S \rightarrow S_\varepsilon$.

Pour chaque variable X , on déterminera (grâce à un algorithme que nous présenterons dans la suite) si $L(X_\varepsilon)$ est égal à $\{\varepsilon\}$ ou à l'ensemble vide $\emptyset$. Si $L(X_\varepsilon) = \{\varepsilon\}$ (resp. $L(X_\varepsilon) = \emptyset$), on remplacera X_ε par ε (resp. $\emptyset$) dans les membres droits de toutes les règles possibles. On détruira

ensuite toutes les règles où une variable du type X_ε apparaît à gauche. Le système ainsi obtenu possède les mêmes solutions que le système de départ, mais ε n'apparaît plus sauf éventuellement dans la règle $S \rightarrow \varepsilon$. De plus S n'apparaît dans le membre droit d'aucune règle.

Reprenons l'exemple que nous avons utilisé jusqu'à présent. On vérifie que les solutions minimales correspondantes sont $L(L_\varepsilon) = \varepsilon$ et $L(N_{1\varepsilon}) = L(N_{2\varepsilon}) = L(A_\varepsilon) = L(B_\varepsilon) = \emptyset$. Substituons alors ces valeurs aux variables intervenant dans les règles données par la méthode que nous venons de voir. $N_2 \rightarrow LB$ se transforme alors en les quatre règles suivantes:

$$\begin{aligned} N_{2+} &\rightarrow L_+ V_{b+} \\ N_{2+} &\rightarrow L_+ V_{b\varepsilon} \\ N_{2+} &\rightarrow L_\varepsilon V_{b+} \\ N_{2\varepsilon} &\rightarrow L_\varepsilon V_{b\varepsilon} , \end{aligned}$$

qui elles-même (une fois les substitutions réalisées), deviennent

$$\begin{aligned} N_{2+} &\rightarrow L_+ V_{b+} \\ N_{2+} &\rightarrow V_{b+} . \end{aligned}$$

On peut en effet détruire les deux autres règles car leurs seconds membres sont $\emptyset$. La grammaire complète transformée est donnée ci-dessous:

$$\begin{aligned} L &\rightarrow \varepsilon \\ L &\rightarrow L_+ \\ L_+ &\rightarrow V_{a+} \\ L_+ &\rightarrow V_{a+} N_{1+} \\ N_{1+} &\rightarrow L_+ V_{a+} \\ N_{1+} &\rightarrow V_{a+} \\ L_+ &\rightarrow V_{b+} \\ L_+ &\rightarrow V_{b+} N_{2+} \\ N_{2+} &\rightarrow L_+ V_{b+} \\ N_{2+} &\rightarrow V_{b+} \\ V_{a+} &\rightarrow a \\ V_{b+} &\rightarrow b . \end{aligned}$$

Comme promis, nous allons maintenant présenter un algorithme qui détermine pour toute variable X si $L(X_\varepsilon)$ est égal à $\emptyset$ ou à $\{\varepsilon\}$. Considérons alors les règles qui font uniquement intervenir des variables de la forme X_ε . Nous pouvons déterminer toutes les solutions correspondantes: il suffit d'essayer toutes les solutions possibles (il y en a 2^k à tester s'il y a k telles variables) et de voir lesquelles conviennent. Si $X_\varepsilon = \emptyset$ est une solution possible pour la variable X_ε , on a $L(X_\varepsilon) = \emptyset$ (on cherche les solutions minimales), sinon on a $L(X_\varepsilon) = \{\varepsilon\}$. Un algorithme plus rapide est décrit à l'Annexe A.

1.7 Elimination des règles unitaires

Rappelons qu'une règle est dite *unitaire* si elle est de la forme $A \rightarrow B$ où A et B sont deux variables.

On dira qu'une variable A conduit à une variable B si $A = B$ ou s'il existe des règles $A \rightarrow B_1, B_1 \rightarrow B_2, \dots, B_{k-1} \rightarrow B_k$ avec $B_k = B$. Déterminons pour toute paire de variables X_0, X_1 si X_0 conduit à X_1 (un algorithme résolvant ce problème est donné en Annexe B).

Pour chaque règle $X \rightarrow c$ où c est une lettre et pour chaque variable X_0 qui conduit à X , introduisons la règle $X_0 \rightarrow c$.

Pour chaque règle du type $X \rightarrow YZ$ où Y, Z sont des variables et pour chaque variable X_0 qui conduit à X , chaque variable Y_1 (resp. Z_1) telle que Y (resp. Z) conduit à Y_1 (resp. Z_1), introduisons la règle $X_0 \rightarrow Y_1Z_1$ (cela revient à tenir compte des règles unitaires dont est issue X et à voir quelles sont les variables que l'on peut obtenir par une règle unitaire à partir de Y et de Z).

Dans l'exemple avec lequel nous avons travaillé jusqu'à présent, on vérifiera que la règle $L_+ \rightarrow V_{a+}N_{1+}$ donne naissance par cette méthode aux quatre règles:

$$\begin{aligned} L_+ &\rightarrow V_{a+}N_{1+} \\ L_+ &\rightarrow V_{a+}V_{a+} \\ L &\rightarrow V_{a+}N_{1+} \\ L &\rightarrow V_{a+}V_{a+} . \end{aligned}$$

On élimine enfin toutes les règles unitaires. Le lecteur vérifiera que cette transformation ne modifie pas les solutions de notre système. Dans notre exemple, la grammaire qui en résultera sera alors

$$\begin{aligned} L &\rightarrow \varepsilon \\ L_+ &\rightarrow V_{a+}N_{1+} \\ L_+ &\rightarrow V_{a+}V_{a+} \\ L &\rightarrow V_{a+}N_{1+} \\ L &\rightarrow V_{a+}V_{a+} \\ N_{1+} &\rightarrow L_+V_{a+} \\ N_{1+} &\rightarrow V_{a+}V_{a+} \\ N_{1+} &\rightarrow V_{b+}V_{a+} \\ L_+ &\rightarrow V_{b+}N_{2+} \\ L_+ &\rightarrow V_{b+}V_{b+} \\ L &\rightarrow V_{b+}N_{2+} \\ L &\rightarrow V_{b+}V_{b+} \\ N_{2+} &\rightarrow L_+V_{b+} \\ N_{2+} &\rightarrow V_{a+}V_{b+} \\ N_{2+} &\rightarrow V_{b+}V_{b+} \\ V_{a+} &\rightarrow a \end{aligned}$$

$$\begin{array}{lcl}
L & \rightarrow & a \\
L_+ & \rightarrow & a \\
N_{1+} & \rightarrow & a \\
V_{b+} & \rightarrow & b \\
L & \rightarrow & b \\
L_+ & \rightarrow & b \\
N_{2+} & \rightarrow & b .
\end{array}$$

Chaque règle est maintenant de la forme $X \rightarrow c$ (avec $c \in \Sigma$), $X \rightarrow YZ$ ou $S \rightarrow \varepsilon$, S désignant l'axiome. De plus, S n'apparaît dans le second membre d'aucune règle du système. Si $L(S)$ ne contient pas le mot vide, chaque règle est alors de la forme $X \rightarrow c$ ou $X \rightarrow YZ$. Cette forme normale des grammaires algébriques est appelée la *forme normale de Chomsky*. La *taille* d'une grammaire algébrique G (notée $|G|$) est le nombre total de caractères utilisés dans toutes ses règles. Lorsqu'on transforme une grammaire algébrique G en une grammaire algébrique G' équivalente sous forme normale de Chomsky, le résultat n'est pas tellement plus gros que la grammaire dont on est parti car on a $|G'| = O(|G|)$.

Chapitre 2

L'algorithme CYK

D'après [eRB94], il existe un algorithme pour tester si un mot w appartient à un langage algébrique L . La méthode naïve consiste à chercher un calcul réussi sur un mot w pour un automate non déterministe à pile borné associé à L . Cela nécessite malheureusement de tester un nombre exponentiel de cas et l'algorithme qui en résulte est beaucoup trop lent. Nous allons présenter ici un algorithme bien plus efficace pour résoudre ce problème, l'algorithme CYK.¹ Celui-ci peut de plus être facilement modifié pour produire des arbres de dérivation des mots de L .

Nous pouvons tester si un langage algébrique contient ou non le mot vide d'après l'Annexe A. Nous pouvons donc supposer sans perte de généralité que le langage algébrique avec lequel nous allons travailler ne contient pas le mot vide. Considérons ensuite une grammaire G associée mise sous forme normale de Chomsky. Soit $w = a_1 \cdots a_n$ un mot de longueur n et posons $w_{ik} = a_i \cdots a_k$. L'algorithme que nous allons présenter va déterminer pour chaque i, k (avec $0 < i \leq k \leq n$) et chaque variable X si w_{ik} peut être dérivé à partir de X . Appelons $T[i, k, X]$ la réponse (vrai ou faux) à cette question. Considérons d'abord le cas $i = k$: on a alors $w_{ik} = w_{ii} = c$ où c est une lettre terminale de G . Comme G est sous forme normale de Chomsky, $T[i, i, X]$ sera alors vrai ssi $X \rightarrow c$ est une règle de G . Supposons maintenant que $k > i$ de sorte que $|w_{ik}| \geq 2$. Alors $T[i, k, X]$ est vrai si et seulement si il existe une règle du type $X \rightarrow YZ$ et une décomposition $w_{ik} = uv$ de w_{ik} telles que Yu et Zv . Autrement dit $T[i, k, X]$ est vrai si et seulement si il existe une règle $X \rightarrow YZ$ et un entier j avec $i \leq j < k$ tel que w_{ij} et $w_{j+1, k}$ puissent être respectivement dérivés à partir de Y et de Z . Cela conduit donc à la récurrence suivante:

$$T[i, k, X] = \begin{cases} \text{vrai} & \text{si } i = k \text{ et qu'il existe une règle } X \rightarrow w_{ii} , \\ \text{vrai} & \text{si } i < k \text{ et qu'il existe une règle } X \rightarrow YZ , \\ & \text{telle que } (\exists j)[i \leq j < k, T[i, j, Y] \text{ et } T[j+1, k, Z]] , \\ \text{faux} & \text{sinon.} \end{cases}$$

Cette récurrence peut être facilement résolue en utilisant la récursivité. Cela étant dit, pour être plus efficace, nous utiliserons la technique de programmation dynamique. Nous stockons les valeurs de T dans un tableau dont toutes les entrées sont initialisées à faux. Il faut ensuite parcourir ce tableau de manière à ce que $T[i, j, Y]$ et $T[j+1, k, Z]$ soient évalués avant $T[i, k, X]$ pour tout $i \leq j \leq k$. Une façon de réaliser cet objectif est de parcourir T par ordre croissant de k et par ordre décroissant de i comme le montre l'algorithme suivant:

Procédure CYK(mot w)

¹CYK est l'acronyme de Cocke, Younger et Kasami qui ont inventé une variante de l'algorithme que nous donnons ici.

```

 $n = |w|$ 
Pour toute variable  $X$  de  $V$  Faire
  Pour  $i = 1$  à  $n$  Faire
    Pour  $k = i$  à  $n$  Faire
       $T[i, k, X] = \text{faux}$ 
    Fin pour
  Fin pour
  Pour  $i = 1$  à  $n$  Faire
    Si  $X \rightarrow w_{ii}$  est une règle Alors
       $T[i, i, X] = \text{vrai}$ 
    Fin Si
  Fin pour
Fin pour
Pour  $k = 2$  à  $n$  Faire
  Pour  $i = k - 1$  à  $1$  Faire
    Pour toute règle de la forme  $X \rightarrow Y Z$  Faire
      Pour  $j = i$  à  $k - 1$  Faire
        Si  $T[i, j, Y]$  et  $T[j + 1, k, Z]$  Alors
           $T[i, k, X] = \text{vrai}$ 
        Fin Si
      Fin pour
    Fin pour
  Fin pour
Fin pour

```

L'algorithme CYK prend en entrée un mot w et une grammaire algébrique G (en forme normale de Chomsky) et teste si $w \in L(G)$. Il n'y a bien entendu pas d'algorithme CYK différent pour chaque grammaire.

Chapitre 3

Réalisation

Dans ce chapitre nous expliquons le fonctionnement de notre programme `chomsky` en donnant les algorithmes et les structures de données utilisées.

3.1 Structures de données

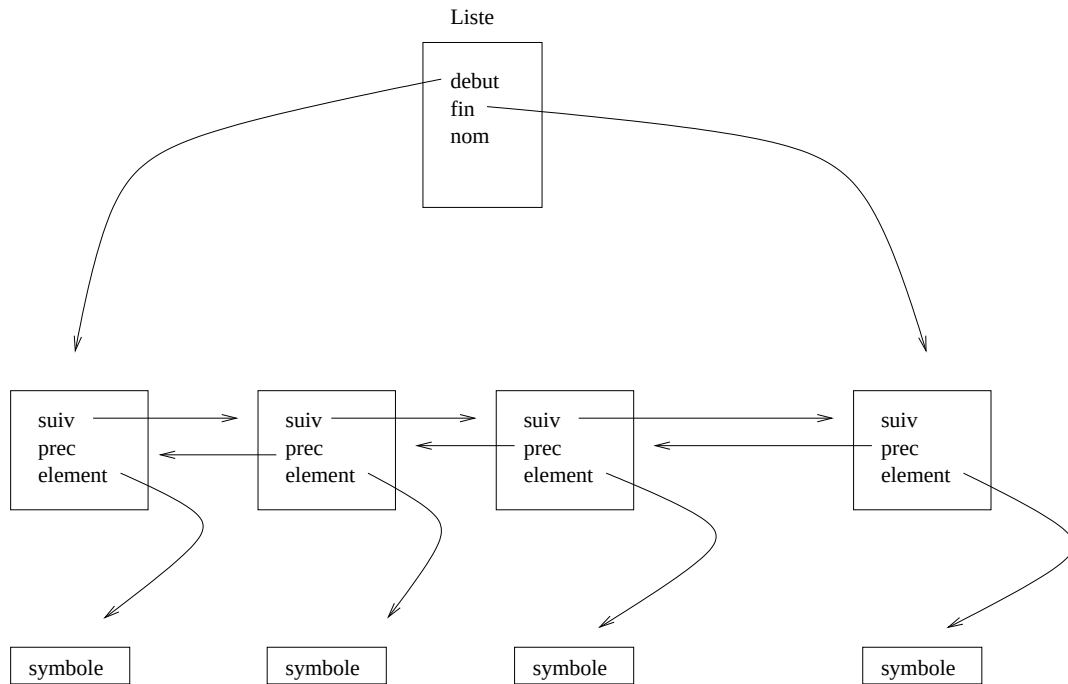
Nous avons décidé de représenter les ensembles par des listes chaînées génériques. Ainsi, les ensembles Σ des symboles terminaux et V des symboles non terminaux, sont des listes d'objets de types `symbole` définis de la façon suivante (cf figure-3.1):

```
typedef struct symbole {
    char *nom;
    unsigned short int num;
    BOOL originel;
} symbole;
```

Pour ce qui est de la grammaire G saisie par l'utilisateur, nous utilisons également une liste dont les éléments sont des objets de type `regle` définis de la façon suivante (cf figure-3.2):

```
typedef struct regle {
    symbole *symb;
    Liste *production;
    unsigned short int num;
} regle;
```

Ce que nous appelons symbole d'une règle r est en fait un pointeur vers un symbole de V et correspond au membre gauche de la règle r . Le membre droit de r est une liste de pointeurs vers des symboles de V ou des symboles de Σ .

Figure 3.1: *Liste de symboles*

3.2 Saisie de la grammaire

La grammaire algébrique G que l'utilisateur désire saisir doit être en fait un système d'équations algébriques où l'on a déjà effectué la distribution, ainsi que le remplacement de $=$ par $\supseteq$. Les symboles terminaux sont définis par l'expression rationnelle

$$E_X = \{a, b, c, \dots, z, +, -, /, *\}.$$

Les symboles non terminaux (appelés *variables* par la suite) sont définis par l'expression rationnelle

$$E_V = \{A, B, \dots, Z\}((\{a, b, \dots, z\} \cup \{A, B, \dots, Z\}) \cup \{0, 1, \dots, 9\})^*.$$

Pour réaliser de manière efficace cette saisie, nous avons utilisé les outils **flex** [JRL95] et **bison** [STA91] afin de créer un module d'analyse syntaxique s'occupant d'initialiser correctement nos listes X , V et G .

La syntaxe de notre langage impose que l'union $\cup$ soit notée $|$, que le langage réduit au mot vide ε soit noté $\#$, et que l'inégalité $\supseteq$ soit notée $\rightarrow$. De plus, les ensembles Σ et V , seront respectivement notés X et V . Enfin, nous noterons les variables de la forme V_c+ par $V_c\sim$ et les variables de la forme N_x par N_x .

Notre programme **chomsky** prend en entrée un fichier texte décrivant le système d'équations algébriques de G et respectant la syntaxe décrite ci-dessus comme le montre l'exemple suivant:

```
% ENSEMBLES
X = { a b }
V = { L }
```

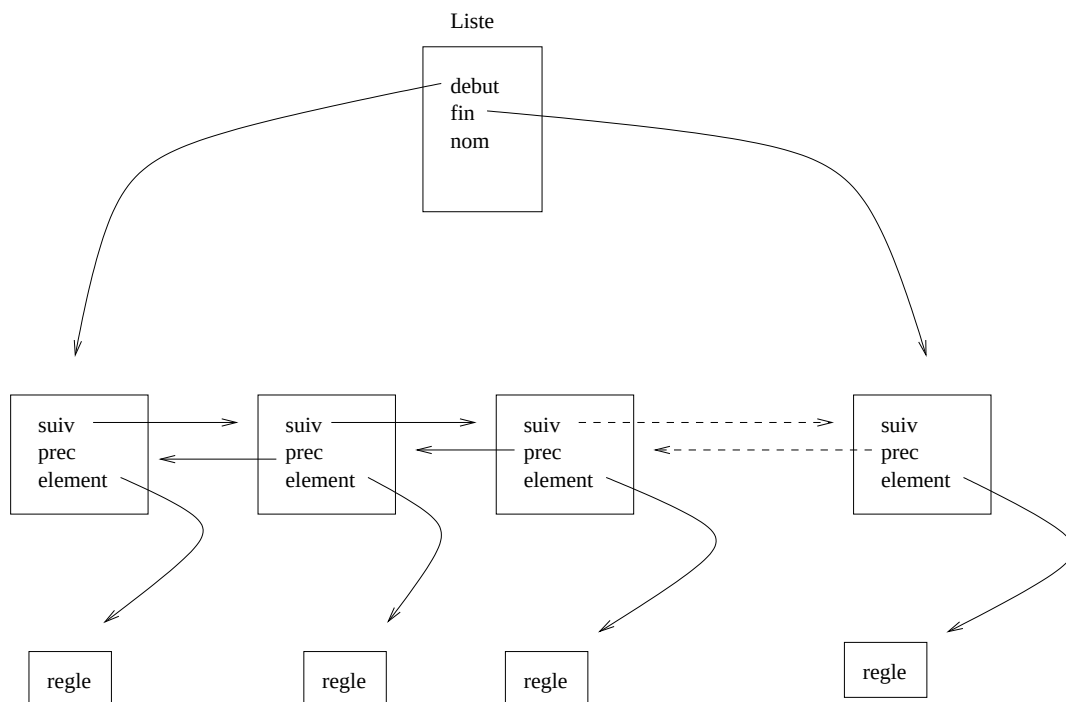


Figure 3.2: Liste de règles

```
% GRAMMAIRE
L -> # | a | a L a | b | b L b
```

Lors de la lecture d'un fichier décrivant une grammaire par notre programme, le module de saisie positionne à vrai le champ `originel` de tous les symboles de type variables. Cela nous servira par la suite afin de déterminer si un symbole non terminal est originel ou non.

Remarque: Pour nous faciliter la tâche de la saisie de la grammaire G , nous avons besoin de décrire en premier lieu les ensembles X et V . Il est possible bien sûr de les déduire de G et donc d'éviter d'entrer la section de description des ensembles, mais là n'est pas le but du projet.

3.3 Standardisation et mise en forme normale de Chomsky de la grammaire G saisie

Nous allons maintenant décrire comment nous avons réalisé la standardisation et la mise en forme normale de Chomsky décrite au Chapitre 1. Nous venons de voir ci-dessus que le système d'équations algébriques saisi par l'utilisateur doit déjà avoir subi les opérations de distribution (cf paragraphe 1.1) et de remplacement de $=$ par $\supseteq$ (cf paragraphe 1.2).

Ainsi, il ne reste donc plus qu'à effectuer les étapes suivantes en séquence:

- Elimination de l'union

- Transformation du membre droit de chaque règle en une lettre ou en une concaténation de variables
- Elimination des membres droits trop longs
- Elimination d'autant de mots vides que possible
- Elimination des règles unitaires

3.3.1 Elimination de l'union

Voici l'algorithme utilisé correspondant à la réalisation de cette étape (cf paragraphe 1.3):

Procédure Etape1(G)

Pour chaque règle r de G **Faire**

 Soit X le membre gauche de r

 Soit l la liste correspondant au membre droit de r

Pour chaque symbole s de l **Faire**

Si s est le caractère | **Alors**

 Insérer une nouvelle règle $r_2 : X \rightarrow$ dans G

 Soit l_2 la liste correspondant au membre droit de r_2

 Supprimer le symbole s de l

Tant que on n'est pas à la fin de la liste l et que le symbole actuel de l n'est pas le caractère | **Faire**

 Insérer dans l_2 le symbole actuel de l

 Supprimer le symbole actuel de l

Fin tant que

Fin Si

Fin pour

Fin pour

3.3.2 Transformation du membre droit de chaque règle en une lettre ou en une concaténation de variables

Voici l'algorithme utilisé correspondant à la réalisation de cette étape (cf paragraphe 1.4):

Procédure Etape2(G)

Pour chaque règle r de G **Faire**

 Soit X le membre gauche de r

 Soit l la liste correspondant au membre droit de r

Si le nom du symbole X ne contient pas le caractère '_' **Alors**

Pour chaque symbole s de l **Faire**

Si le symbole s est terminal **Alors**

 Créer un nouveau nom de symbole $\text{nom} = V_<\text{nom du symbole } s>$

Si un symbole de nom nom n'existe pas déjà dans V **Alors**

 Insérer un nouveau symbole s_2 de nom nom dans V

 Insérer une nouvelle règle de la forme $s_2 \rightarrow s$ dans G

Fin Si

 Remplacer le symbole s de l par s_2

Fin Si
Fin pour
Fin Si
Fin pour

3.3.3 Elimination des membres droits trop longs

Voici l'algorithme utilisé correspondant à la réalisation de cette étape (cf paragraphe 1.5):

Procédure Etape3(G)

$i = 1$

Pour chaque règle r de G **Faire**

Soit p la liste correspondant au membre droit de r

Si NombreElements(p) > 2 **Alors**

Pour chaque symbole $symp$ de p **Faire**

Si il existe un symbole après $symp$ **Alors**

Créer un nouveau nom de symbole $nom = N_i$

$i++$

Insérer un nouveau symbole s de nom nom comme dernier élément de V

Insérer une nouvelle règle $r_2 : s \rightarrow$ dans G

Soit l la liste correspondant au membre droit de r_2

Tant que on n'est pas à la fin de la liste p **Faire**

Insérer le symbole courant de p à la fin de la liste l

Supprimer le symbole courant de p

Fin tant que

Insérer le dernier élément de V en fin de la liste p

Fin Si

Fin pour

Fin Si

Fin pour

3.3.4 Elimination d'autant de mots vides que possible

Voici l'algorithme utilisé correspondant à la réalisation de cette étape qui peut se décomposer pour plus de simplicité en quatre sous-étapes (cf paragraphe 1.6):

Procédure Etape4(G)

Etape4a(G)

Etape4b(G)

Etape4c(G)

Etape4d(G)

La première sous-étape permet d'ajouter les nouveaux symboles non terminaux dans V et d'ajouter les nouvelles règles dans G :

Procédure Etape4a(G)

Pour chaque symbole $symp$ de V **Faire**

Insérer un nouveau symbole v_1 de nom $\langle \text{nom de symb} \rangle \sim$ dans V
 Insérer une nouvelle règle $\text{symb} \rightarrow v_1$ dans G
 Insérer un nouveau symbole v_2 de nom $\langle \text{nom de symb} \rangle \#$ dans V
 Insérer une nouvelle règle $\text{symb} \rightarrow v_2$ dans G

Fin pour

Remarque: Nous ne pouvons pas utiliser le caractère '+' dans la création des nouvelles variables, car il s'agit d'un symbole autorisé dans l'expression rationnelle décrivant les symboles non terminaux. Nous avons choisis de le remplacer par le caractère '~' qui est lui interdit dans le fichier d'entrée.

La seconde sous-étape permet d'effectuer les modifications des règles:

Procédure Etape4b(G)

Pour chaque règle r de G exceptés les règles ajoutées dans Etape4a() **Faire**

Soit s le membre gauche de r

Soit l la liste correspondant au membre droit de r

Soit s_1 le premier symbole de $actu$

Si s_1 est un symbole terminal {Modification du type $X \rightarrow c : X+ \rightarrow c$ } **Alors**

Remplacer la règle $s \rightarrow s_1$ par $s\sim \rightarrow s_1$ dans G

Fin Si

Si s_1 est le symbole ε {Modification du type $X \rightarrow \# : X\# \rightarrow \#$ } **Alors**

Remplacer la règle $s \rightarrow \#$ par $s\# \rightarrow \#$ dans G

Fin Si

Si il n'existe pas de symbole après le symbole s_1 {Modification du type $X \rightarrow Y : X+ \rightarrow Y+, X\# \rightarrow Y\#$ } **Alors**

Remplacer la règle $s \rightarrow s_1$ par les deux règles $s\sim \rightarrow s_1\sim$ et $s\# \rightarrow s_1\#$

Sinon

{Modification du type $X \rightarrow Y Z : X+ \rightarrow Y+ Z+, X+ \rightarrow Y+ Z\#, X+ \rightarrow Y\# Z+, X\# \rightarrow Y\# Z\#$ }

Soit s_2 le second symbole de l

Remplacer la règle $s \rightarrow s_1 s_2$ par les quatre règles $s\sim \rightarrow s_1\sim s_2\sim$, $s\sim \rightarrow s_1\# s_2\sim$, $s\sim \rightarrow s_1\sim s_2\#$, et $s\# \rightarrow s_1\# s_2\#$

Fin Si

Fin pour

La troisième sous-étape permet de supprimer les variables originelles à l'exception de l'axiome:

Procédure Etape4c(G)

Pour chaque règle r de G **Faire**

Soit s le membre gauche de r

Si s n'est pas l'axiome de G **Alors**

Si s est une variable originelle **Alors**

Supprimer la règle r de G

Fin Si

Fin Si

Fin pour

Supprimer toutes les variables de V qui ne sont pas utilisées dans G

La dernière étape, de loin la plus complexe de tout le projet, détermine pour toute variable $X \in V$ si $L(X\#) = 0$ ou si $L(X\#) = \#$. Le traitement à effectuer dans ces deux cas est décrit au

paragraphe 1.6.

Procédure Etape4d(G)

Initialiser le tableau BESOINS

Pour chaque variable X de V **Faire**

Chercher dans V la variable $X\#$

Si $L(X\#) = \text{VIDE}$ **Alors**

Pour chaque règle r de G contenant $X\#$ dans son membre droit **Faire**

Supprimer le symbole $X\#$ du membre droit de r

Fin pour

Sinon Si $L(X\#) = \text{EPSILON}$ **Alors**

Pour chaque règle r de G contenant $X\#$ dans son membre droit **Faire**

Remplacer le symbole $X\#$ du membre droit de r par $\#$

Si tous les symboles du membre droit de r sont $\#$ **Alors**

Supprimer la règle r de G

Fin Si

Fin pour

Fin Si

Fin pour

Pour chaque règle r de G de la forme $X\# \rightarrow \dots$ restantes **Faire**

Supprimer r de G

Fin pour

Supprimer toutes les variables de V qui ne sont pas utilisées dans G

Remarque: La partie délicate de cette sous-étape provient de la fonction $L()$ utilisée. Cette fonction nécessite l'existence d'un tableau nommé BESOINS qui est initialisé au début de la procédure Etape4d(). La création du tableau BESOINS ainsi que le contenu de la fonction $L()$ sont expliqués en Annexe A.

3.3.5 Elimination des règles unitaires

Voici l'algorithme utilisé correspondant à la réalisation de cette étape (cf paragraphe 1.7):

Procédure Etape5(G)

Numérotage des règles de la grammaire G

Numérotage des symboles de la liste V

Initialiser T , la matrice de fermeture transitive de la relation $\rightarrow$

{Pour chaque règle de la forme $X \rightarrow c$ }

Pour chaque règle r de G de la forme $X \rightarrow c$ **Faire**

Pour chaque variable X_0 de V telle que $\text{Conduit}(X_0, X)$ et que les numéros des symboles X_0 et X soient différents **Faire**

Insérer une nouvelle règle $X_0 \rightarrow c$ dans G

Fin pour

Fin pour

Créer trois nouvelles listes tmp_0, tmp_1, tmp_2

{Pour chaque règle de la forme $X \rightarrow Y Z$ }

Pour chaque règle r de G de la forme $X \rightarrow Y Z$ **Faire**

Pour chaque variable X_0 de V **Faire**

Si $\text{Conduit}(X_0, X)$ **Alors**

```

    Insérer  $X_0$  dans  $tmp_0$ 
Fin Si
Si Conduit(  $Y$ ,  $X_0$  ) Alors
    Insérer  $X_0$  dans  $tmp_1$ 
Fin Si
Si Conduit(  $Z$ ,  $X_0$  ) Alors
    Insérer  $X_0$  dans  $tmp_2$ 
Fin Si
Pour chaque triplet  $(s_0, s_1, s_2)$  du produit cartésien  $tmp_0 \times tmp_1 \times tmp_2$  Faire
    Insérer une nouvelle règle  $s_0 \rightarrow s_1 s_2$  dans  $G$ 
Fin pour
Fin pour
Fin pour
{Pour chaque règle unitaire}
Pour chaque règle  $r$  de  $G$  de la forme  $A \rightarrow B$  Faire
    Supprimer  $r$  de  $G$ 
Fin pour

```

Remarque: Dans cette étape, nous utilisons une fonction nommée `Conduit()`. Cette fonction nécessite l'utilisation de la matrice T , initialisée au début de l'algorithme ci-dessus. La création de la matrice T ainsi que le contenu de la fonction `Conduit()` sont donnés en Annexe B.

3.4 Algorithme CYK

Cet algorithme détermine si un mot w appartient à $L(G)$, le langage engendré par la grammaire G saisie par l'utilisateur. L'algorithme est donné au Chapitre 2.

3.5 Utilisation

L'utilisation de notre programme `chomsky` est très simple. L'aide peut d'ailleurs être obtenue en utilisant l'option `--help` sur la ligne de commande: `chomsky --help` qui affiche alors le message suivant:

```

Usage: chomsky -i <input file> [-o <output file>] [--help] [?] [-v]
    Converts the input grammar file into chomsky grammar form.
Options:
    -o output file: save the chomsky grammar in to file 'output file'.
    -v: verbose mode.
    -help | ?: displays this text.

```

L'option `-i` permet d'indiquer le nom du fichier (<input file>) décrivant la grammaire G à utiliser. L'option `-o` permet de sauvegarder la forme normale de G dans le fichier indiqué au paramètre <output file>. L'option `-v` permet d'afficher le détail des différentes étapes de la mise en forme normale. Une fois la grammaire mise en forme normale, le programme demandera alors à l'utilisateur de saisir un mot w . Un appel à `CYK(  $w$  )` répondra alors si oui ou non le mot w appartient à $L(G)$.

Remarque: Avant de lancer `CYK()` proprement dit, nous effectuons une pré-analyse de w afin de savoir si tous les symboles terminaux de w appartiennent bien à l'alphabet terminal X indiqué dans le fichier `<input file>`. Si tel n'est pas le cas, il est bien évident que l'appel à `CYK( w )` est inutile puisque $w \notin L(G)$ dans ce cas.

Conclusion

Ce projet très intéressant nous aura permis de nous familiariser avec les langages algébriques, utilisés en théorie des langages et en compilation. Nous avons particulièrement apprécié le double aspect théorique et pratique de ce projet, qui même s'il ne nous sera pas d'une grande utilité, aura cependant le mérite de nous avoir montré comment fonctionnent des outils comme les compilateurs, ou comme `bison`, ou `yacc`.

Si un regret devait être exprimé, ce serait celui de n'avoir pu ajouter, faute de temps, quelques améliorations à notre programme. Citons en effet la possibilité d'avoir des terminaux de longueur variable dans la grammaire saisie. Une autre amélioration serait d'éviter d'avoir à saisir les ensembles V et Σ dans le fichier d'entrée. Mais nous avons jugé que cela ne faisait pas partie des priorités du projet et n'avons plus eu le temps de les ajouter.

Enfin, nous sommes également fiers d'avoir pu utiliser nos structures de liste doublement chaînée génériques ainsi que de matrices génériques, développées quelques mois avant ce projet. Cela nous a permis de gagner du temps et de nous concentrer sur les détails du projet plutôt que sur les structures de données citées ci-dessus.

Annexe A

Algorithme déterminant si

$$\forall V, \varepsilon \in L(V)$$

Nous allons maintenant voir l'algorithme utilisé pour déterminer pour toute variable V d'une grammaire algébrique si $\varepsilon \in L(V)$. La complexité de cet algorithme est linéaire en la taille de la grammaire G . Nous allons d'abord décrire rapidement la solution. Nous aurons besoin pour cela d'un peu de terminologie propre à cette partie. Nous dirons qu'une règle dépend de toutes les variables de son membre de droite et qu'une variable X dépend de toutes les règles de la forme $X \rightarrow Y_1 \cdots Y_k$. Nous allons maintenant définir récursivement la notion d'utilisabilité: nous dirons donc qu'une règle est utilisable lorsque toutes les variables dont elle dépend sont utilisables: une règle de la forme $X \rightarrow \varepsilon$ sera donc immédiatement utilisable; nous dirons de même qu'une variable est utilisable lorsqu'au moins une des règles dont elle dépend est utilisable. Nous désignerons maintenant sous le nom d'objet, soit une règle, soit une variable. Nous dirons qu'une règle du type $X \rightarrow Y_1 \cdots Y_k$ a besoin de k objets et qu'une variable X a besoin d'un seul objet. Lorsqu'un objet α devient utilisable pour la première fois, nous regardons quels sont les autres objets qui dépendent de α : leurs besoins sont alors réduits d'une unité et le traitement de l'objet α est terminé. Si le traitement d'une variable est terminé, on peut en dériver le mot vide. En effectuant cette méthode dans le bon ordre, nous éviterons de traiter deux fois un objet.

Pour chaque objet α , **DependDe** $[\alpha]$ contiendra la liste des objets dont α dépend. Le tableau **DependDe** $[\]$ peut être facilement calculé en temps linéaire en la taille de la grammaire. Le reste de l'algorithme est donné ci-dessous. Le mot vide ε peut se dériver à partir d'une variable X si et seulement si X appartient à **ObjetsTraités**.

Il est utile de penser que les objets forment les sommets d'un graphe orienté. Il y aura une arête reliant l'objet α à l'objet β si α dépend de β . Etiquetons les variables par "OU", les règles vides par "VRAI" et toutes les autres règles par "ET". Ce graphe peut être considéré comme un circuit booléen (qui peut contenir des cycles). Supposons que toutes les portes "ET" et "OU" démarrent dans l'état "FAUX". L'algorithme que nous avons décrit ci-dessous donne l'état stable atteint par chaque porte booléenne. Cet algorithme peut être étendu à des circuits booléens à seuils (une porte à seuil ne peut être franchie que si au moins t de ses entrées sont égales à "VRAI"; on initialisera alors l'entrée correspondante de **BESOINS** à t).

Procédure Construire le tableau **BESOINS**

ObjetsUtilisables = $\emptyset$

```

ObjetsTraités =  $\emptyset$ 
Pour toute variable  $X$  Faire
    BESOINS[ $X$ ] = 1
Fin pour
Pour toute règle  $P$  Faire
    BESOINS[ $P$ ] = la longueur du membre droit de  $P$ 
    Si BESOINS[ $P$ ]  $\leq 0$  { $P$  est de la forme  $X \rightarrow \varepsilon$ } Alors
        Insérer  $P$  dans ObjetsUtilisables
    Fin Si
Fin pour
Tant que ObjetsUtilisables  $\neq \emptyset$  Faire
    Oter un objet  $\alpha$  {une variable ou une règle} de ObjetsUtilisables
    Si  $\alpha \notin$  ObjetsTraités Alors
        Insérer  $\alpha$  dans ObjetsTraités
        Pour chaque objet  $\beta$  de DependDe[ $\alpha$ ] Faire
            BESOINS[ $\beta$ ] = BESOINS[ $\beta$ ] - 1
            Si BESOINS[ $\beta$ ]  $\leq 0$  Alors
                Insérer  $\beta$  dans ObjetsUtilisables
            Fin Si
        Fin pour
    Fin Si
Fin tant que

```

Voici maintenant le contenu de la fonction $L()$ que nous utilisons dans la procédure **Etape4d()** du paragraphe 3.3.4.

```

Procédure  $L(\text{symbole } s)$ 
Si BESOINS[ $s$ ] = 0 Alors
    Retourner EPSILON
Sinon
    Retourner VIDE
Fin Si

```

Annexe B

Algorithme déterminant si une variable A conduit à une variable B

Rappelons que pour la réalisation de l'étape 5 de la mise en forme normale de Chomsky (voir paragraphe 1.7) nous avons besoin de savoir pour tout couple de variables X_0, X_1 si X_0 conduit à X_1 . Pour cela, il est nécessaire de calculer la fermeture transitive de la relation $\rightarrow$. Comme cette relation peut être représentée par un graphe que nous noterons $\mathcal{G}$, nous la représenterons en machine par une matrice d'adjacence M booléenne. La matrice $M[n][n]$ sera une matrice carrée dont le nombre d'éléments en ligne et en colonne n sera égal au nombre de variables de l'ensemble V , nous avons donc $n = |V| = \text{nombre de sommets du graphe } \mathcal{G}$.

Pour initialiser la matrice M , nous commençons par mettre à 0 tous ses éléments sauf sur sa diagonale que nous mettons à 1. Ensuite, nous parcourons les règles de la grammaire G . Pour chaque règle de la forme $X_0 \rightarrow X_1$, nous mettons à 1 la case (X_0, X_1) de la matrice M . Ensuite nous créons une nouvelle matrice T qui est une copie de M et qui contiendra le résultat de la fermeture transitive de M .

Procédure Initialisation de M et T

Pour $i = 1$ à n **Faire**

Pour $j = 1$ à n **Faire**

Si $i = j$ **Alors**

$M[i][j] = 1$

$T[i][j] = 1$

Sinon

$M[i][j] = 0$

$T[i][j] = 0$

Fin Si

Fin pour

Fin pour

Pour toute règle r de G de la forme $X_0 \rightarrow X_1$ **Faire**

 Soient x_0 et x_1 les numéros respectifs des symboles X_0 et X_1

$M[x_0][x_1] = 1$

$T[x_0][x_1] = 1$

Fin pour

Il reste maintenant à calculer T , la matrice de la fermeture transitive de M . Pour cela, rappelons que s'il existe un chemin entre deux sommets du graphe $\mathcal{G}$, il existe un chemin élémentaire entre ces sommets, de longueur $< n$: la matrice de la clôture transitive est donc la somme logique

$$M \oplus M^2 \oplus \dots \oplus M^{n-1}.$$

Mais on peut calculer la matrice T de la clôture transitive de façon plus rapide en utilisant l'algorithme de Warshall ci-dessous:

L'algorithme ci-dessous résume la construction de T :

Procédure Fermeture transitive de Warshall(G)

Pour $i = 1$ à n **Faire**

Pour $j = 1$ à n **Faire**

Si $T[j][i] = 1$ **Alors**

Pour $k = 1$ à n **Faire**

Si $T[i][k] = 1$ **Alors**

$T[j][k] = 1$

Fin Si

Fin pour

Fin Si

Fin pour

Fin pour

Le principe de l'algorithme est le suivant: pour chaque sommet i , on examine tous les sommets j et k : s'il existe un chemin de j à i et un chemin de i à k , on en déduit qu'il existe un chemin de j à k .

Nous pouvons maintenant décrire la fonction **Conduit()** utilisée au paragraphe 3.3.5:

Procédure Conduit(symbole s_1 , symbole s_2)

Soient n et m les numéros respectifs des symboles s_1 et s_2

Si $T[n][m] > 0$ **Alors**

Retourner TRUE

Sinon

Retourner FALSE

Fin Si

Annexe C

Quelques exemples

Dans ce chapitre, nous présentons quelques exemples que nous avons utilisés avec succès sur notre programme `chomsky`.

C.1 Exemple a: le langage ETF

C.1.1 Fichier décrivant la grammaire

```
% ENSEMBLES
X = { + * n ) ( }
V = { Expression Terme Facteur }

% GRAMMAIRE
Expression -> Expression + Terme | Terme
Terme -> Terme * Facteur | Facteur
Facteur -> ( Expression ) | n
```

C.1.2 Forme normale de Chomsky

```
R1 : Expression~ -> Expression~ N_1~
R2 : Expression~ -> V_n~ N_1~
R3 : Expression~ -> Facteur~ N_1~
R4 : Expression~ -> Terme~ N_1~
R5 : Expression -> V_n~ N_1~
R6 : Expression -> Facteur~ N_1~
R7 : Expression -> Terme~ N_1~
R8 : Expression -> Expression~ N_1~
R9 : N_1~ -> V_+~ Terme~
R10 : N_1~ -> V_+~ V_n~
R11 : N_1~ -> V_+~ Facteur~
R12 : Terme~ -> Terme~ N_2~
```

```

R13 : Terme~ -> V_n~ N_2~
R14 : Terme~ -> Facteur~ N_2~
R15 : Expression~ -> V_n~ N_2~
R16 : Expression~ -> Facteur~ N_2~
R17 : Expression~ -> Terme~ N_2~
R18 : Expression -> V_n~ N_2~
R19 : Expression -> Facteur~ N_2~
R20 : Expression -> Terme~ N_2~
R21 : N_2~ -> V_*~ Facteur~
R22 : N_2~ -> V_*~ V_n~
R23 : Facteur~ -> V_(~ N_3~
R24 : Terme~ -> V_(~ N_3~
R25 : Expression~ -> V_(~ N_3~
R26 : Expression -> V_(~ N_3~
R27 : N_3~ -> Expression~ V_)~
R28 : N_3~ -> V_n~ V_)~
R29 : N_3~ -> Facteur~ V_)~
R30 : N_3~ -> Terme~ V_)~
R31 : V_+~ -> +
R32 : V_*~ -> *
R33 : V_(~ -> (
R34 : V_)~ -> )
R35 : Facteur~ -> n
R36 : Terme~ -> n
R37 : Expression~ -> n
R38 : Expression -> n
R39 : V_n~ -> n

```

C.2 Exemple b: le langage des mots de parenthèses sur $\{a, b\}$

C.2.1 Fichier décrivant la grammaire

```

% ENSEMBLES
X = { a b }
V = { S }

% GRAMMAIRE
S -> a S b S | #

```

C.2.2 Forme normale de Chomsky

```

R1 : S~ -> V_a~ N_1~
R2 : S~ -> V_a~ N_2~
R3 : S~ -> V_a~ V_b~
R4 : S -> V_a~ N_2~
R5 : S -> V_a~ N_1~
R6 : S -> V_a~ V_b~

```

```

R7  : N_1~ -> S~ N_2~
R8  : N_1~ -> S~ V_b~
R9  : N_2~ -> V_b~ S~
R10 : N_1~ -> V_b~ S~
R11 : V_a~ -> a
R12 : N_2~ -> b
R13 : N_1~ -> b
R14 : V_b~ -> b
R15 : S -> #

```

C.3 Exemple c: une variante du langage des mots de parenthèses sur $\{a, b\}$

C.3.1 Fichier décrivant la grammaire

```

% ENSEMBLES
X = { a b }
V = { S }

% GRAMMAIRE
S -> a S b S | a | b | #

```

C.3.2 Forme normale de Chomsky

```

R1  : S~ -> V_a~ N_1~
R2  : S~ -> V_a~ N_2~
R3  : S~ -> V_a~ V_b~
R4  : S -> V_a~ N_2~
R5  : S -> V_a~ N_1~
R6  : S -> V_a~ V_b~
R7  : N_1~ -> S~ N_2~
R8  : N_1~ -> V_b~ N_2~
R9  : N_1~ -> V_b~ V_b~
R10 : N_1~ -> V_a~ N_2~
R11 : N_1~ -> V_a~ V_b~
R12 : N_1~ -> S~ V_b~
R13 : N_2~ -> V_b~ S~
R14 : N_2~ -> V_b~ V_b~
R15 : N_2~ -> V_b~ V_a~
R16 : N_1~ -> V_b~ V_b~
R17 : N_1~ -> V_b~ V_a~
R18 : N_1~ -> V_b~ S~
R19 : S~ -> a
R20 : S -> a
R21 : V_a~ -> a
R22 : N_2~ -> b

```

```

R23 : N_1~ -> b
R24 : S~ -> b
R25 : S -> b
R26 : V_b~ -> b
R27 : S -> #

```

C.4 Exemple d: le langage des palindromes sur $\{a, b\}$

C.4.1 Fichier décrivant la grammaire

```

% ENSEMBLES
X = { a b }
V = { L }

% GRAMMAIRE
L -> # | a | a L a | b | b L b

```

C.4.2 Forme normale de Chomsky

```

R1  : L~ -> V_a~ N_1~
R2  : L~ -> V_a~ V_a~
R3  : L -> V_a~ N_1~
R4  : L -> V_a~ V_a~
R5  : N_1~ -> L~ V_a~
R6  : N_1~ -> V_b~ V_a~
R7  : N_1~ -> V_a~ V_a~
R8  : L~ -> V_b~ N_2~
R9  : L~ -> V_b~ V_b~
R10 : L -> V_b~ N_2~
R11 : L -> V_b~ V_b~
R12 : N_2~ -> L~ V_b~
R13 : N_2~ -> V_b~ V_b~
R14 : N_2~ -> V_a~ V_b~
R15 : N_1~ -> a
R16 : L~ -> a
R17 : L -> a
R18 : V_a~ -> a
R19 : N_2~ -> b
R20 : L~ -> b
R21 : L -> b
R22 : V_b~ -> b
R23 : L -> #

```

Bibliographie

- [AA91] Jeffrey ULLMAN Alfred AHO, Ravi SETHI. *Compilateurs, Principes, techniques et outils*. InterEditions, Paris, 1991.
- [eRB94] Robert FLOYD et Richard BIEGEL. *Le langage des machines*. International Thomson Publishing France, Paris, 1994.
- [JRL95] Doug BROWN John R. LEVINE, Tony MASON. *Lex & Yacc*. Editions O'Reilly International Thomson, 1, rue Saint Georges, 75009 Paris, 1995.
- [STA91] Charles DONNELLY & Richard M. STALLMAN. *Bison, the YACC-compatible Parser Generator*. Free Software Foundation, December 1991.